

Algorithmique

Arbres de recherche (rouge-noir)

Cédric Lhoussaine

Université de Lille / IUT de Lille

» Outline

Rappels sur les arbres de recherche

Arbres rouge-noir

Insertion dans les arbres RN

Suppression dans les arbres RN

» Arbres binaires

- * un arbre binaire est un arbre dont tous les noeuds sont d'arité au plus 2

» Arbres binaires

- * un arbre binaire est un arbre dont tous les noeuds sont d'arité au plus 2
- * un arbre binaire de hauteur h possède, au plus,

» Arbres binaires

- * un arbre binaire est un arbre dont tous les noeuds sont d'arité au plus 2
- * un arbre binaire de hauteur h possède, au plus, $2^{h+1} - 1$ noeuds
(rappel : la hauteur d'un arbre est le nombre maximal d'arcs qui relient la racine à une feuille).

» Arbres binaires

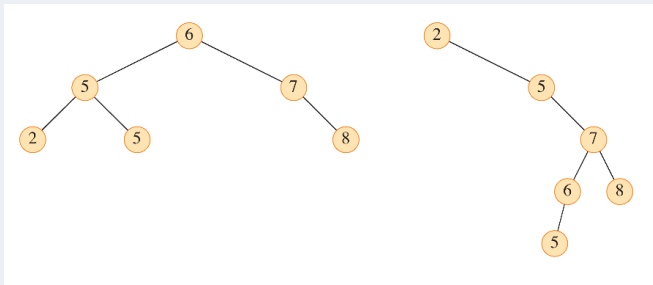
- * un arbre binaire est un arbre dont tous les noeuds sont d'arité au plus 2
- * un arbre binaire de hauteur h possède, au plus, $2^{h+1} - 1$ noeuds (rappel : la hauteur d'un arbre est le nombre maximal d'arcs qui relie la racine à une feuille).
- * un arbre binaire **complet** de hauteur h est un arbre qui possède, **exactement**, $2^{h+1} - 1$ noeuds.

» Arbres binaires de recherche (ABR)

Un **arbre binaire de recherche** (ABR) est un arbre binaire dont chaque noeud possède une *clé* et qui satisfait la propriété suivante : soit x un noeud, si y est un noeud du sous-arbre gauche de x , alors $y.cle \leq x.cle$; si y est noeud du sous-arbre droit de x , alors $y.cle \geq x.cle$.

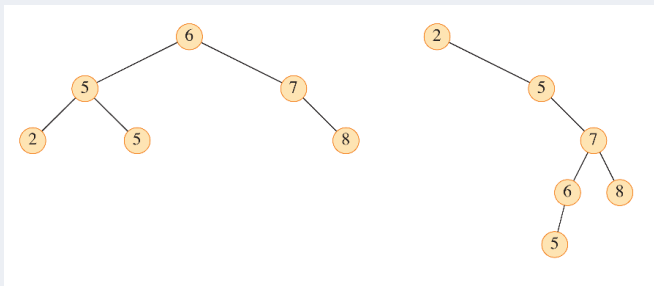
» Arbres binaires de recherche (ABR)

Un **arbre binaire de recherche** (ABR) est un arbre binaire dont chaque noeud possède une *clé* et qui satisfait la propriété suivante : soit x un noeud, si y est un noeud du sous-arbre gauche de x , alors $y.cle \leq x.cle$; si y est noeud du sous-arbre droit de x , alors $y.cle \geq x.cle$.



» Arbres binaires de recherche (ABR)

Un **arbre binaire de recherche** (ABR) est un arbre binaire dont chaque noeud possède une *clé* et qui satisfait la propriété suivante : soit x un noeud, si y est un noeud du sous-arbre gauche de x , alors $y.cle \leq x.cle$; si y est noeud du sous-arbre droit de x , alors $y.cle \geq x.cle$.



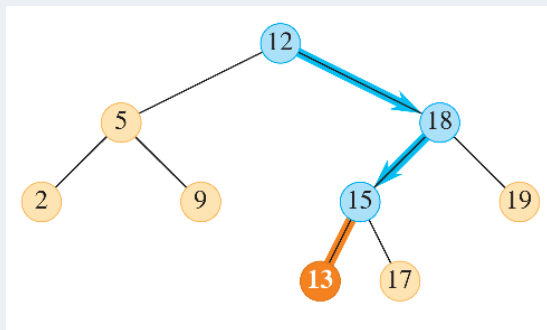
⚠ Ne pas confondre tas et ABR (aucun des deux n'est un cas particulier de l'autre !)

» Recherche dans un ABR

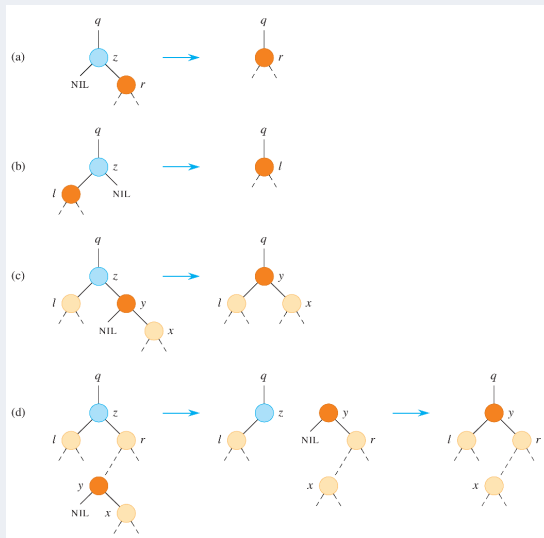
```
function recherche( $x, k$ )  
  if  $x == NIL$  or  $k == x.cle$  then  
    | return  $x$   
  if  $k < x.cle$  then  
    | return recherche( $x.gauche, k$ )  
  else  
    | return recherche( $x.droite, k$ )
```

» Insertion de z

Recherche (binaire) de la bonne position pour z + insertion de z comme feuille.
Exemple : insertion de 13



» Suppression de z



» Coût des opérations sur les ABR

- * Dans le pire cas, les opérations sur les ABR sont en $\Theta(h)$ où h est la hauteur de l'arbre.

» Coût des opérations sur les ABR

- * Dans le pire cas, les opérations sur les ABR sont en $\Theta(h)$ où h est la hauteur de l'arbre.
- * Mais, il peut arriver que $h = n$ nombre total de noeud...

» Coût des opérations sur les ABR

- * Dans le pire cas, les opérations sur les ABR sont en $\Theta(h)$ où h est la hauteur de l'arbre.
- * Mais, il peut arriver que $h = n$ nombre total de noeud...
- * Le coût optimal $\Theta(\log(n))$ est obtenu pour les ABR complets !

» Coût des opérations sur les ABR

- * Dans le pire cas, les opérations sur les ABR sont en $\Theta(h)$ où h est la hauteur de l'arbre.
- * Mais, il peut arriver que $h = n$ nombre total de noeud...
- * Le coût optimal $\Theta(\log(n))$ est obtenu pour les ABR complets !
- * $\Rightarrow$ Nécessité de structures un peu plus complexes pour que les insertions et suppressions gardent l'ABR à peu près complet : arbres rouge-noir.

» Arbres rouge-noir

Un **arbre rouge-noir** est un ABR dont les noeuds contiennent, en plus des attributs *cle*, *gauche*, *droite* et *p* (père), **un attribut couleur (= rouge ou noir)**. De plus, il satisfait les propriétés suivantes :

1. chaque noeud est soit rouge, soit noir
2. la racine et les feuilles sont noires
3. si un noeud est rouge alors ses enfants sont noirs
4. pour chaque sous-arbre, toutes les branches reliant la racine de ce sous-arbre à ses feuilles ont le même nombre de noeuds noirs.

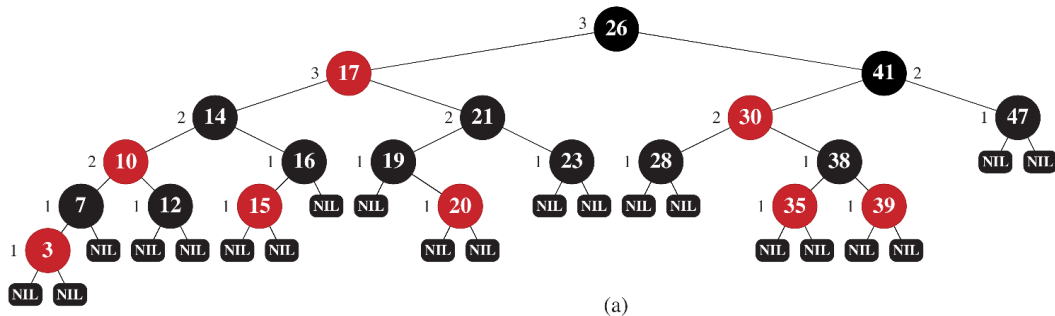
» Arbres rouge-noir

Un **arbre rouge-noir** est un ABR dont les noeuds contiennent, en plus des attributs *cle*, *gauche*, *droite* et *p* (père), **un attribut couleur (= rouge ou noir)**. De plus, il satisfait les propriétés suivantes :

1. chaque noeud est soit rouge, soit noir
2. la racine et les feuilles sont noires
3. si un noeud est rouge alors ses enfants sont noirs
4. pour chaque sous-arbre, toutes les branches reliant la racine de ce sous-arbre à ses feuilles ont le même nombre de noeuds noirs.

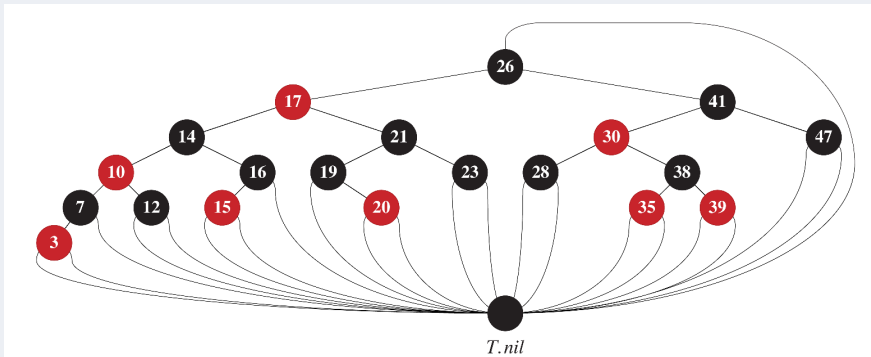
$hn(x)$ = nb de noeud noirs de x à une de ses feuilles (x exclus)

» Arbres rouge-noir



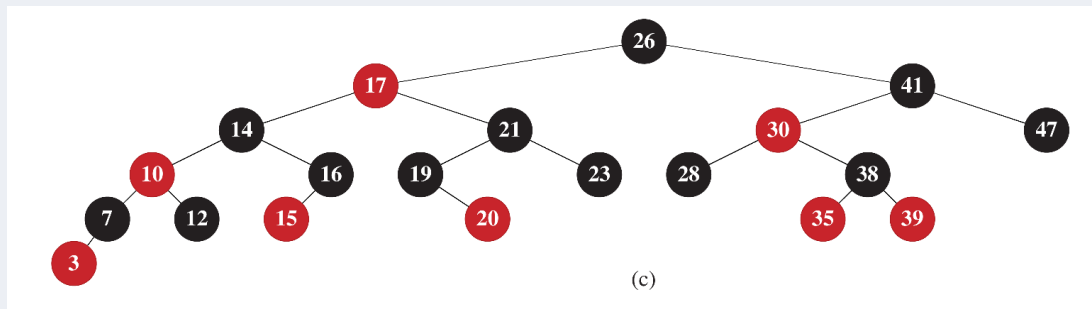
» Arbres rouge-noir

En pratique, on utilisera une sentinelle partagée ($T.nil$) qui sert à la fois à repérer les feuilles et la racine (ie. unique noeud ayant $T.nil$ pour père) :



» Arbres rouge-noir

Dans les exemples graphiques, la sentinelle ne sera pas représentée :



» Arbres rouge-noir

Propriété

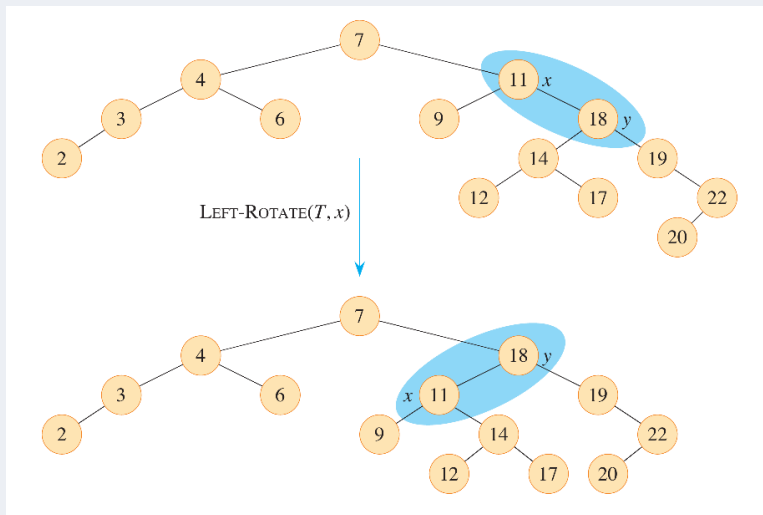
Un arbre rouge-noir ayant n noeuds internes a une hauteur au plus égale à $2 \log(n + 1)$.

» Arbres rouge-noir

Les insertions et suppressions doivent préserver les propriétés d'arbre rouge-noir $\Rightarrow$ réorganisation des noeuds après chaque insertion/suppression.

» Rotations

Première transformation



» Rotations

On remarque que les rotations

- * préservent la propriété d'ABR (i.e. les noeuds sont bien ordonnés)

» Rotations

On remarque que les rotations

- * préservent la propriété d'ABR (i.e. les noeuds sont bien ordonnés)
- * peuvent diminuer la hauteur du sous-arbre en rotation (si la rotation est effectuée dans le bon sens)

» Rotations

On remarque que les rotations

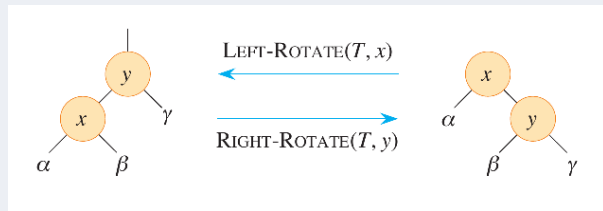
- * préservent la propriété d'ABR (i.e. les noeuds sont bien ordonnés)
- * peuvent diminuer la hauteur du sous-arbre en rotation (si la rotation est effectuée dans le bon sens)
- * mais peuvent briser la contrainte de coloriage (i.e. les noeuds rouges ne peuvent pas avoir de fils rouges)

» Rotations

```

1 : function Rotation-gauche( $T, x$ )
2 :    $y \leftarrow x.droite$ 
3 :    $x.droite \leftarrow y.gauche$ 
4 :   if  $y.gauche \neq T.nil$  then
5 :      $y.gauche.p \leftarrow x$ 
6 :    $y.p \leftarrow x.p$ 
7 :   if  $x.p == T.nil$  then
8 :      $T.racine \leftarrow y$ 
9 :   else if  $x == x.p.gauche$  then
10 :     $x.p.gauche \leftarrow y$ 
11 :   else
12 :     $x.p.droite \leftarrow y$ 
13 :    $y.gauche \leftarrow x$ 
14 :    $x.p \leftarrow y$ 

```



» Insertion

function Insérer(T, z)

$y \leftarrow T.nil$

▷ mémorise le père de x

$x \leftarrow T.racine$

while $x \neq T.nil$ **do**

▷ recherche la position d'insertion

$y \leftarrow x$

if $z.cle < x.cle$ **then**

$x \leftarrow x.gauche$

else

$x \leftarrow x.droite$

$z.p \leftarrow y$

▷ y devient père de z

if $y == T.nil$ **then**

 ▷ Si y est la racine

$T.racine \leftarrow z$

▷ alors z devient la racine

else if $z.cle < y.cle$ **then**

▷ sinon il faut insérer z en fils gauche ou droit de y

$y.gauche \leftarrow z$

else

$y.droite \leftarrow z$

▷ z est une feuille rouge

$z.gauche \leftarrow T.nil$

$z.droite \leftarrow T.nil$

$z.couleur \leftarrow \text{rouge}$

Insérer-correction(T, z)

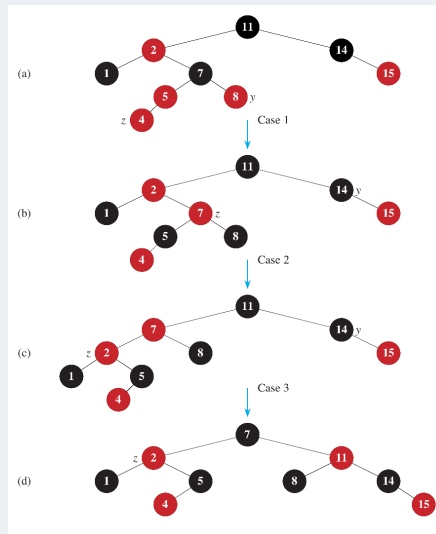
▷ rééquilibrage de l'arbre en arbre rouge-noir

» Rééquilibrage rouge-noir

```

function Insérer-correction(T, z)
  while z.p.couleur == rouge do
    if z.p == z.p.p.gauche then
      y ← z.p.p.droite
      if y.couleur == rouge then
        z.p.couleur ← noir
        y.couleur ← noir
        z.p.p.couleur ← rouge
        z ← z.p.p
      else
        if z == z.p.droite then
          z ← z.p
          rotation-gauche(T, z)
          z.p.couleur ← noir
          z.p.p.couleur ← rouge
          rotation-droite(T, z.p.p)
        else
          (idem "droite" ↔ "gauche"...)
  T.racine.couleur ← noir
  
```

▷ cas 1
 ▷ cas 1
 ▷ cas 1
 ▷ cas 1
 ▷ cas 2
 ▷ cas 2
 ▷ cas 3
 ▷ cas 3
 ▷ cas 3



» Complexité de l'insertion

Dans le pire cas, l'insertion est en $O(\log(n))$.

» Transplantation

Transplante le noeud v à la place du noeud u

```
function Transplantation( $T, u, v$ )
    if  $u.p == T.nil$  then
        |    $T.racine \leftarrow v$ 
    else if  $u == u.p.gauche$  then
        |    $u.p.gauche \leftarrow v$ 
    else
        |    $u.p.droite \leftarrow v$ 
     $v.p \leftarrow u.p$ 
```

» Suppression

```
function Supprimer(T, z)
```

```
  y ← z
```

```
  y_couleur_originale ← y.couleur
```

```
  if z.gauche == T.nil then
```

```
    | x = z.droite
```

```
    | Transplante(T, z, z.droite)
```

```
  else if z.droite == T.nil then
```

```
    | x ← z.gauche
```

```
    | Transplante(T, z, z.gauche)
```

```
  else
```

```
    | y ← Minimum(z.droite)
```

```
    | y_couleur_originale ← y.couleur
```

```
    | x ← y.droite
```

```
  if y == z.droite then
```

```
    | x.p ← y
```

```
  else
```

```
    | Transplante(T, y, y.droite)
```

```
    | y.droite ← z.droite
```

```
    | y.droite.p ← y
```

```
    | Transplante(T, z, y)
```

```
    | y.gauche ← z.gauche
```

```
    | y.gauche.p ← y
```

```
    | y.couleur ← z.couleur
```

```
  if y_couleur_originale == NOIRE then
```

```
    | Supprimer_correction(T, x)
```

» Rééquilibrage rouge-noir

```

function Supprimer-correction(T, x)
  while x ≠ T.racine and x.couleur == NOIR do
    if x == x.p.gauche then
      w ← x.p.droite
      if w.couleur == ROUGE then
        w.couleur ← NOIR
        x.p.couleur ← ROUGE
        rotation-gauche(T, x.p)
      w ← x.p.droite
      if w.gauche.couleur == NOIR and
        w.droite.couleur == NOIR then
        w.couleur ← ROUGE
        x ← x.p
      else

```

```

      if w.droite.couleur == NOIR then
        w.gauche.couleur ← NOIR
        w.couleur ← ROUGE
        rotation-droite(T, w)
      w ← x.p.droite
      w.couleur ← x.p.couleur
      x.p.couleur ← NOIR
      w.droite.couleur ← NOIR
      rotation-gauche(T, x.p)
      x ← T.racine
    else
      (idem branche "then" avec "droite" ↔
       "gauche"...)
    x.couleur ← NOIR

```

Emacs 30.2 (Org mode 9.7.31)